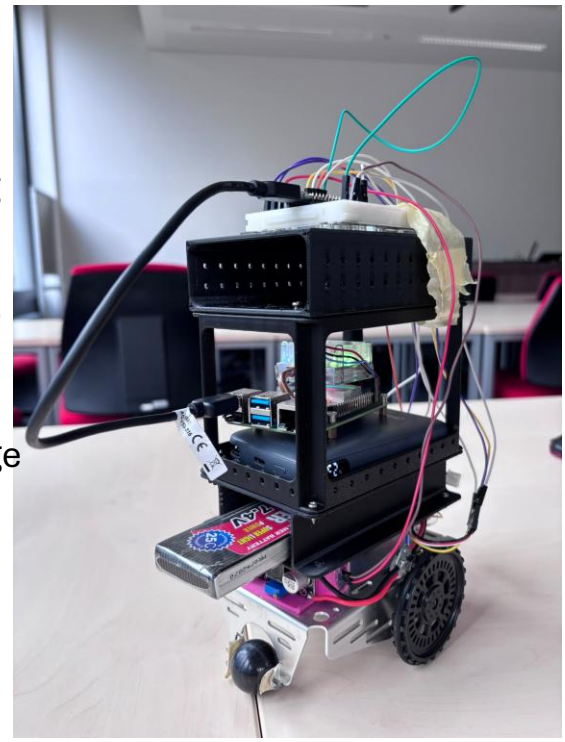
RASPBERRY PI REMOTE ACCESS

AND ROS 2 ROBOT EXPERIMENT

CONTENTS

1. Overview and message architecture
2. Connect RASPBERRY PI remotely using SSH
3. Optional graphical desktop using XRDP
4. Install and verify ROS 2
5. Create a workspace and Python package
6. Run sensor and controller nodes
7. Inspect, echo and publish topics
8. Connect physical hardware safely
9. Run the experiment and record results



1.OVERVIEW AND ARCHITECTURE

ROS 2 lets separate programs (nodes) exchange typed messages through named topics.

A sensor publishes a measurement; a controller subscribes that measurement and publishes a command; a motor interface subscribes to the command and translates it into hardware-specific signals.

NODE	TOPIC	POURPOSE
Sensor node	Publishes /scan	Publishes distance, in meters in this example
Controller node	Subscribes /scan; publishes /cmd_vel	Slows down and commands zero near the wall
Motor Interface	Subscribes /cmd_vel	Maps commands to a real motor driver
Monitor node	Subscribes to any topic	Shows messages for debugging

2. Connect RASPBERRY PI remotely using SSH

To configure and control your robot without a dedicated monitor, establish an SSH (Secure Shell) connection:

2.1 Ensure both your computer and the Raspberry Pi are connected to the same Wi-Fi network. Open a terminal (macOS/Linux) or Command Prompt/PowerShell (Windows).

Execute the following command to connect (replace with your actual username and hostname/IP address):

3. Optional graphical desktop using XRDP

XRDP lets a Windows computer open a Linux desktop session.

It is optional: ROS 2 works from SSH terminals without a graphical desktop.

3.1 Install XFCE and XRDP on Ubuntu Server

sudo apt update

sudo apt install xrdp xfce4 xfce4-session –y

```
sudo adduser xrdp ssl-cert
echo xfce4-session > ~/.xsession
sudo systemctl enable --now xrdp
sudo systemctl restart xrdp
sudo systemctl status xrdp
```

Session behavior differs by release, if login fails, check the XRDP logs and desktop-session configuration.

3.2 Connect from Windows

1. Find the Pi IP address with hostname -I.

2. Press Win + R on Windows, enter mstsc, and press Enter.

3. Enter the Pi IP address and choose Connect.

4. At the XRDP login screen, select Xorg if offered and enter your Linux username and password.

Hardware & Robot Construction

Based on the experiment configuration, the setup contains the following core components:

- SBC Node: Raspberry Pi serving as the high-level brain running the primary ROS2 environment.
- Microcontroller Node: A breadboarded microcontroller development board (such as an ESP32 or STM32) acting as an embedded real-time bridging interface.
- Power Subsystem: External battery bank providing regulated voltage rails through an XT60 or equivalent high-amperage connector block to isolate motor noise from logic loops.
- Signal Interconnects: Serial UART lines configured via jumper pins connecting the microcontroller's TX/RX nodes directly to the Raspberry Pi's GPIO pins

4. Install and verify ROS 2

4.1 Configure the ROS 2 apt repository and using the following commands.

```
sudo apt update
sudo apt install software-properties-common curl -y
sudo add-apt-repository universe -y
sudo apt update
sudo curl sSLhttps://raw.githubusercontent.com/ros/rosdistro/master/ros.key \ -o /usr/share/keyrings/ros-archive-keyring.gpg
echo"deb [arch=$(dpkg --print architecture) signed by=/usr/share/keyrings/ros-archive-keyring.gpg] http://packages.ros.org/ros2/ubuntu $(. /etc/os-release && echo $UBUNTU_CODENAME) main" | sudo tee /etc/apt/sources.list.d/ros2.list > /dev/null

sudo apt update
```

4.2 Install ROS 2 and build tools

```
sudo apt install ros-humble-ros-base python3-colcon-common-extensions python3-rosdep -y
sudo rosdep init rosdep update
```

4.3 Source ROS 2 in each terminal

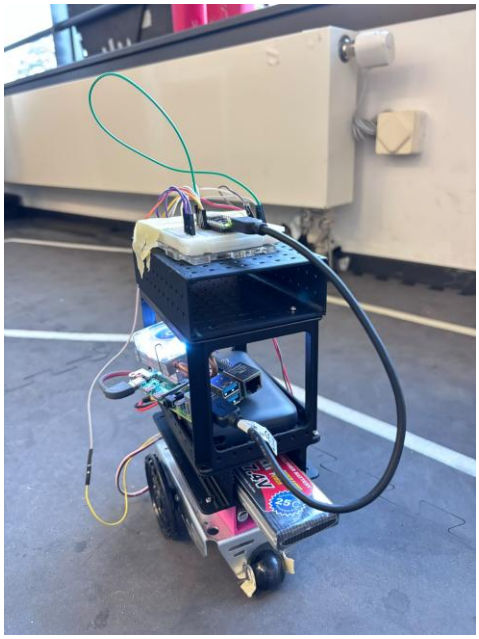
```
echo "source /opt/ros/humble/setup.bash" >> ~/.bashrc
source /opt/ros/humble/setup.bash printenv ROS_DISTRO
ros2 –help
```

- The ROS_DISTRO output should be humble. Open a new terminal or source the setup file again after changing shell configuration.

4.4 Verify with a publisher and subscriber In
For working of robo
Terminal -1

```
cd ~/ros2_ws
colcon build --packages-select serial_node
```

source install/setup.bash



Terminal-2

source /opt/ros/iron/setup.bash
source ~/ros2_ws/install/setup.bash
ros2 run serial_node serial_node_ACM0

Terminal-3

source /opt/ros/iron/setup.bash
source ~/ros2_ws/install/setup.bash
ros2 topic info /kher_serial_ACM0 --verbose
ros2 topic echo /kher_serial_ACM0

Terminal-4

source /opt/ros/iron/setup.bash
source ~/ros2_ws/install/setup.bash
ros2 topic pub --once /kher_serial_ACM0 std_msgs/msg/String "{data: 'ffff'}"

The listener should print messages. Stop each program with Ctrl+C. If the demo package is not installed, install the relevant demo nodes package or continue to the custom package steps below.

5. Create a workspace and Python package

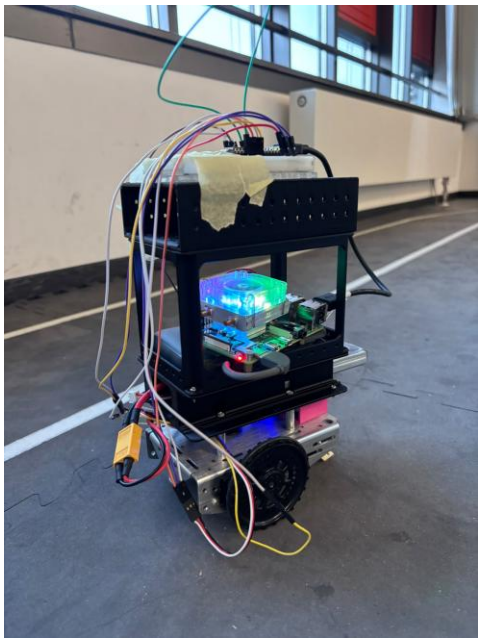
5.1 A workspace stores ROS 2 packages.

source /opt/ros/humble/setup.bash
mkdir -p ~/ros2_ws/src
cd ~/ros2_ws/src
ros2 pkg create --build-type ament_python robot_experiment --dependencies rclpy std_msgs
cd ~/ros2_ws/src/robot_experiment/robot_experiment

nano sensor_node.py

5.2 Add executable entry points in setup.py
Open ~/ros2_ws/src/robot_experiment/setup.py. Find its existing entry_points section and include the following entries; preserve the rest of the generated file.

```
entry_points={
        'console_scripts': [
            'sensor_node=                    robot_experiment.sensor_node:main',
'controller_node = robot_experiment.controller_node:main',
      ],
},
```



5.3 Build and source the workspace
cd ~/ros2_ws
source /opt/ros/humble/setup.bash
colcon build --symlink-install
source install/setup.bash

6. Run sensor and controller nodes

6.1 Terminal A — simulated sensor
source /opt/ros/humble/setup.bash
source ~/ros2_ws/install/setup.bash
ros2 run robot_experiment sensor_node

6.2 Terminal B — controller
source /opt/ros/humble/setup.bash
source ~/ros2_ws/install/setup.bash

ros2 run robot_experiment controller_node

As the distance decreases, the controller publishes a positive speed while distance is above 0.20 m, then publishes zero at or below the threshold. This command is not connected to physical motors.

- **CPP PUBLISHER**

```
#include <chrono>

 #include <functional>

 #include <memory>

 #include <string>

#include "rclcpp/rclcpp.hpp"

 #include "std_msgs/msg/string.hpp"

using namespace std::chrono_literals;

/* This example creates a subclass of Node and uses std::bind() to register a member function as a callback from the timer. */

class MinimalPublisher : public

rclcpp::Node




{

 public:
```

```
MinimalPublisher() :

Node("cpp_publisher"), count_(0)

{

publisher_ = this-

>create_publisher<std_msgs::msg::String>

("cpp_topic", 10);

 timer_=this->create_wall_timer(500ms,
std::bind(&MinimalPublisher::timer_callback, this)); }

private: void timer_callback() { auto message = std_msgs::msg::String();
message.data = "Hello Utkarsh & Kripal cpp: " + std::to_string(count_++);
RCLCPP_INFO(this->get_logger(), "Publishing: '%s'", message.data.c_str());
publisher_->publish(message); } rclcpp::TimerBase::SharedPtr timer_;
rclcpp::Publisher<std_msgs::msg::String>::SharedPtr publisher_; size_t
count_; };

int main(int argc, char * argv[]) { rclcpp::init(argc, argv);
rclcpp::spin(std::make_shared()); rclcpp::shutdown(); return 0; }
```

- **CPP Subscriber**

```
#include <functional>
```

```
#include <memory>

#include "rclcpp/rclcpp.hpp"
#include "std_msgs/msg/string.hpp"

using std::placeholders::_1;

class MinimalSubscriber : public rclcpp::Node
{
public:
  MinimalSubscriber()
  : Node("cpp_subscriber")
  {
    python_sub_ = this->create_subscription<std_msgs::msg::String>(
      "python_topic", 10,  std::bind(&MinimalSubscriber::python_callback, this, _1));
    cpp_sub_ = this->create_subscription<std_msgs::msg::String>(
      "cpp_topic", 10, std::bind(&MinimalSubscriber::cpp_callback, this, _1));
  }

private:
```

```
  void python_callback(const std_msgs::msg::String & msg) const

  {

    RCLCPP_INFO(this->get_logger(), "[from python_topic]I heard: '%s'", msg.data.c_str());

  }


  void cpp_callback(const std_msgs::msg::String & msg) const

  {

    RCLCPP_INFO(this->get_logger(), "[from cpp_topic]I heard: '%s'", msg.data.c_str());

  }

  rclcpp::Subscription<std_msgs::msg::String>::SharedPtr python_sub_;

  rclcpp::Subscription<std_msgs::msg::String>::SharedPtr cpp_sub_;


};


int main(int argc, char * argv[])

{

  rclcpp::init(argc, argv);

  rclcpp::spin(std::make_shared<MinimalSubscriber>());
```

```
 rclcpp::shutdown();

 return 0;

}
```

- **Python Publisher**

```
import rclpy from rclpy.node import Node

from std_msgs.msg import String

class MinimalPublisher(Node):

def                                                    _init_(self):
    super()._init_('Python_publisher')
    self.publisher_      =      self.create_publisher(String,
'python_topic',                                                10)
    timer_period       =       0.5              #       seconds
    self.timer        =        self.create_timer(timer_period,
self.timer_callback)
    self.i                          =                          0

def                                       timer_callback(self):
    msg                         =                         String()
    msg.data  =  'Hello  Utkarsh  &  Kripal  Py:  %d'  %  self.i
    self.publisher_.publish(msg)
    self.get_logger().info('Publishing:   "%s"'   %   msg.data)
    self.i                           +=                         1


def main(args=None): rclpy.init(args=args)

minimal_publisher             =             MinimalPublisher()

rclpy.spin(minimal_publisher)
```

```
#        Destroy        the        node        explicitly
# (optional - otherwise it will be done automatically
# when the garbage collector destroys the node object)
minimal_publisher.destroy_node()
rclpy.shutdown()
```

if *name* == '*main*': main()

- **Python Subscriber**

import rclpy from rclpy.node import Node

from std_msgs.msg import String

class MinimalSubscriber(Node):

```
def                                          _init_(self):
    super()._init_('python_subscriber')
    self.sub_python       =       self.create_subscription(
        String,
        'python_topic',
        self.python_callback,
        10)
    self.sub_cpp = self.create_subscription(  # prevent unused
variable                                                warning
        String,
        'cpp_topic',
        self.cpp_callback,
        10)

def               python_callback(self,               msg):
    self.get_logger().info('[from    python_topic]I    heard:
"%s"'                        %                       msg.data)

def                 cpp_callback(self,                 msg):
```

```
    self.get_logger().info('[from cpp_topic]I heard: "%s"' %
msg.data)

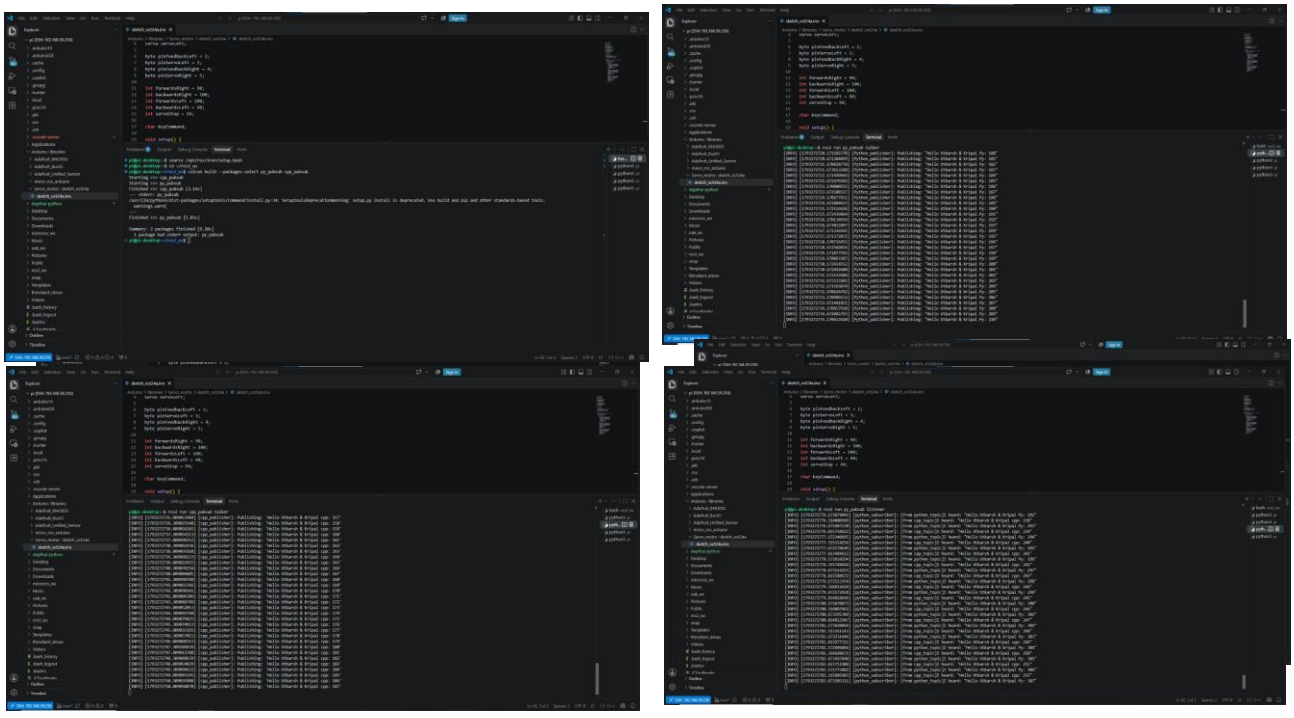

def main(args=None): rclpy.init(args=args)

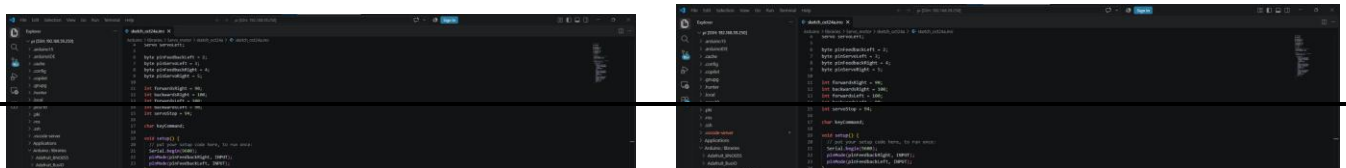
minimal_subscriber = MinimalSubscriber()

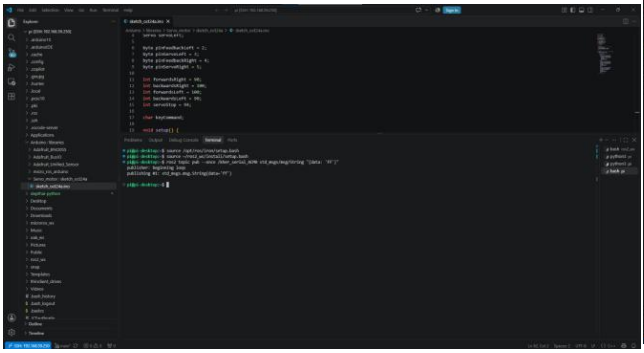
rclpy.spin(minimal_subscriber)

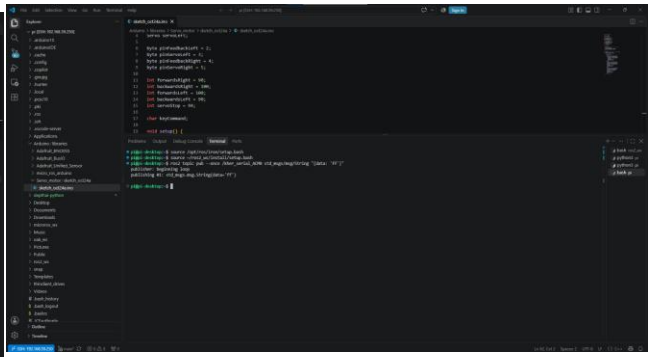
# Destroy the node explicitly
# (optional - otherwise it will be done automatically
# when the garbage collector destroys the node object)
minimal_subscriber.destroy_node()
rclpy.shutdown()

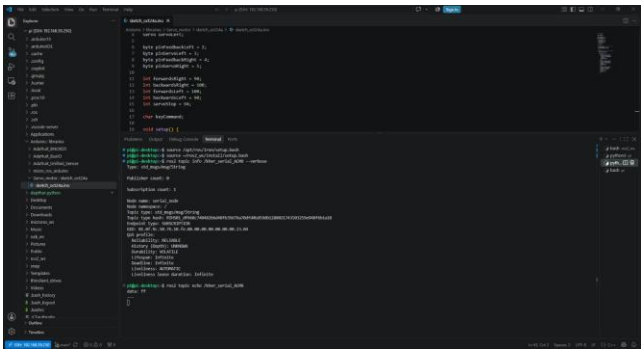

if name == 'main': main()
```











7. Inspect, erobot experimenttopics

7.1 Open a third terminal and source the environment:
source /opt/ros/humble/setup.bash
ros2 topic list
ros2 topic list -t
ros2 topic info /scan
ros2 topic info /cmd_vel
ros2 topic echo /scan

Use Ctrl+C to stop topic echo. To watch /cmd_vel, run ros2 topic echo /cmd_vel in a separate terminal. Topic echo is just another subscriber; it does not remove messages from other subscribers.

7.2 Inspect nodes and types

ros2 node list
ros2 node info /controller_node
ros2 topic info /scan
ros2 topic info /cmd_vel

8. Connect physical hardware safely

1. Identify each board and component by its printed model number. Find the official pinout or datasheet for the sensor, controller, motor driver and battery.
2. Disconnect motor power. Power the Raspberry Pi from its approved power supply.

3. Connect the sensor's power, ground and signal lines according to its interface (GPIO, I^2C, SPI, UART or USB). Verify signal voltage compatibility; Pi GPIO is 3.3 V.

4. Test the sensor separately and confirm the readings and units before creating a ROS 2 driver.

5. Check motor-driver logic voltage, motor supply range, current rating, enable/standby pins and the motor's stall current.

6. Wire the motor driver only after checking its pinout. Follow the driver's grounding requirements.

7. Keep wheels raised for the first motor tests. Start at low speed and short duration, with a physical power disconnect within reach.

8. Replace the simulated sensor publisher with a real sensor driver that publishes correctly scaled distance values.

9. Create a motor-interface node that subscribes to /cmd_vel and maps the command to driver-specific outputs. Add a command timeout and stop output if commands become stale.

10. Test failure cases: sensor disconnected, ROS node stopped, no messages arriving, and emergency stop activated

9. Run the experiment and record results

1. Confirm SSH login works.
2. Confirm ROS_DISTRO shows the intended distribution in every terminal.
3. Run the talker/listener demo.
4. Build the robot_experiment package without errors.
5. Run sensor_node and controller_node in separate terminals.
6. Watch /scan and /cmd_vel with ros2 topic echo.
7. Send a manual 0.15 m /scan message and verify that the controller commands zero.
8. Test the real sensor with motor power disconnected.
9. Perform a low-speed motor test with wheels raised and a physical disconnect available.
10. Write down observed values, errors and changes made.